

```

In [1]: # =====
# ADALINE (Adaptive Linear Neuron) Implementation from Scratch
# =====

# Import NumPy Library
# NumPy is used for array handling, mathematical operations,
# dot products, and numerical computations
import numpy as np

# =====
# Define the ADALINE Class
# =====
class Adaline:

    # -----
    # Constructor Method
    # -----
    def __init__(self, learning_rate=0.01, epochs=50):
        """
        learning_rate : Controls the step size in gradient descent
        epochs          : Number of times the training data is iterated
        """

        # Store learning rate as a class variable
        self.learning_rate = learning_rate

        # Store number of epochs
        self.epochs = epochs

    # -----
    # Fit Method (Training the ADALINE Model)
    # -----
    def fit(self, X, y):
        """
        X : Input feature matrix (n_samples x n_features)
        y : Target output vector
        """

        # Get number of samples and number of features from X
        n_samples, n_features = X.shape

        # Initialize weights to zero for each feature
        # One weight corresponds to one input feature
        self.weights = np.zeros(n_features)

        # Initialize bias term to zero
        self.bias = 0

        # Initialize a list to store loss values for each epoch
        self.losses = []

        # ----- Training Loop -----
        for epoch in range(self.epochs):

            # Compute linear combination (net input)
            # net_input = X · w + b

```

```

linear_output = np.dot(X, self.weights) + self.bias

# Apply linear activation function (identity function)
# ADALINE uses a linear activation
y_predicted = linear_output

# Compute error (difference between actual and predicted output)
errors = y - y_predicted

# Update weights using gradient descent rule
#  $w = w + \eta * (X^T \cdot \text{error}) / n_{\text{samples}}$ 
self.weights += self.learning_rate * np.dot(X.T, errors) / n_samples

# Update bias using gradient descent
#  $b = b + \eta * \text{sum}(\text{error}) / n_{\text{samples}}$ 
self.bias += self.learning_rate * np.sum(errors) / n_samples

# Compute Mean Squared Error (MSE)
# Loss function minimized by ADALINE
loss = (errors ** 2).mean() / 2

# Store Loss for this epoch
self.losses.append(loss)

# Print Loss value for each epoch (optional for observation)
print(f"Epoch {epoch + 1}/{self.epochs}, Loss: {loss:.4f}")

# -----
# Predict Method
# -----
def predict(self, X):
    """
    Predict class labels for input data
    """

    # Compute linear output using trained weights and bias
    linear_output = np.dot(X, self.weights) + self.bias

    # Convert continuous output into class labels
    # If output  $\geq 0 \rightarrow$  class +1
    # Else  $\rightarrow$  class -1
    return np.where(linear_output >= 0, 1, -1)

# =====
# Main Program (Testing ADALINE)
# =====

# Define a simple linearly separable dataset
# Each row represents one data point with two features
X = np.array([
    [ 1,  1],
    [ 1, -1],
    [-1,  1],
    [-1, -1]
])

# Define corresponding target labels
# Classes are represented as +1 and -1

```

```
y = np.array([1, -1, -1, -1])

# Create an object of the Adaline class
# Learning_rate controls how fast the model learns
# epochs define number of training iterations
model = Adaline(learning_rate=0.01, epochs=50)

# Train the ADALINE model using the fit method
model.fit(X, y)

# Predict class labels for the training data
predictions = model.predict(X)

# Display predicted and actual labels
print("\nPredicted Labels:", predictions)
print("Actual Labels   :", y)

# Display final weights and bias
print("\nFinal Weights:", model.weights)
print("Final Bias   :", model.bias)
```

Epoch 1/50, Loss: 0.5000  
Epoch 2/50, Loss: 0.4925  
Epoch 3/50, Loss: 0.4852  
Epoch 4/50, Loss: 0.4781  
Epoch 5/50, Loss: 0.4710  
Epoch 6/50, Loss: 0.4641  
Epoch 7/50, Loss: 0.4574  
Epoch 8/50, Loss: 0.4508  
Epoch 9/50, Loss: 0.4443  
Epoch 10/50, Loss: 0.4379  
Epoch 11/50, Loss: 0.4317  
Epoch 12/50, Loss: 0.4256  
Epoch 13/50, Loss: 0.4196  
Epoch 14/50, Loss: 0.4138  
Epoch 15/50, Loss: 0.4080  
Epoch 16/50, Loss: 0.4024  
Epoch 17/50, Loss: 0.3969  
Epoch 18/50, Loss: 0.3915  
Epoch 19/50, Loss: 0.3862  
Epoch 20/50, Loss: 0.3810  
Epoch 21/50, Loss: 0.3759  
Epoch 22/50, Loss: 0.3709  
Epoch 23/50, Loss: 0.3660  
Epoch 24/50, Loss: 0.3612  
Epoch 25/50, Loss: 0.3565  
Epoch 26/50, Loss: 0.3519  
Epoch 27/50, Loss: 0.3474  
Epoch 28/50, Loss: 0.3429  
Epoch 29/50, Loss: 0.3386  
Epoch 30/50, Loss: 0.3343  
Epoch 31/50, Loss: 0.3302  
Epoch 32/50, Loss: 0.3261  
Epoch 33/50, Loss: 0.3221  
Epoch 34/50, Loss: 0.3182  
Epoch 35/50, Loss: 0.3143  
Epoch 36/50, Loss: 0.3106  
Epoch 37/50, Loss: 0.3069  
Epoch 38/50, Loss: 0.3033  
Epoch 39/50, Loss: 0.2997  
Epoch 40/50, Loss: 0.2962  
Epoch 41/50, Loss: 0.2928  
Epoch 42/50, Loss: 0.2895  
Epoch 43/50, Loss: 0.2862  
Epoch 44/50, Loss: 0.2830  
Epoch 45/50, Loss: 0.2799  
Epoch 46/50, Loss: 0.2768  
Epoch 47/50, Loss: 0.2738  
Epoch 48/50, Loss: 0.2708  
Epoch 49/50, Loss: 0.2679  
Epoch 50/50, Loss: 0.2650

Predicted Labels: [ 1 -1 -1 -1]  
Actual Labels : [ 1 -1 -1 -1]

Final Weights: [0.19749697 0.19749697]  
Final Bias : -0.19749696643123169

In [ ]: